

Zig-zag

Given a string, define a *subsequence* of the string to be a non-empty string obtained by deleting 0 or more characters from the original string. For example, "abc" has the subsequences "a", "b", "c", "ab", "ac", "bc", and "abc".

Consider a string S . Let S_i denote the i^{th} character of the string. Moreover, a string S is said to be *zig-zag* if $S_1 < S_2 > S_3 < S_4 > \dots$ or $S_1 > S_2 < S_3 > S_4 < \dots$ (where the order $<$ is given by $a < b < c < \dots < z$, the same order as the English alphabet). So "epal" is a zig-zag string while "aple" and "mcc" are not. Of the 7 subsequences of "abc" given above, only the subsequence "abc" is not zig-zag.

Two subsequences are considered different if the two sets of indices of the characters deleted to obtain the subsequence are not identical.

For example, "mcccm" have 3 different subsequence "cm", obtained by deleting characters at indices {1, 2, 3}, {1, 2, 4} and {1, 3, 4}. And "mcmc" have 3 different subsequence "mc", obtained by deleting characters at indices {3, 4}, {1, 2}, and {2, 3} (1-indexed).

Remember our friend Fluffy the squirrel? After she got the encoded string from Cipher, it is only natural for her to ask the following question: Given a string S , how many zig-zag subsequence of length K are there?

She has already solved the problem. Can you do the same? As the output can get quite large, output the answer as the remainder when divided by 1000.

Constraints

- Length of $S \leq 20000$,
- $1 \leq K \leq 100$,
- S consists of only lowercase English letters.

Sample Input

```
S = "bcade"
```

```
K = 3
```

Sample Output

```
5
```

Explanation

The 5 zig-zag subsequence of length 3 are "bca", "bad", "bae", "cad", and "cae".

Solution

The case for $k = 1$ is trivial, with answer $|S|$. We will now only consider for $k \neq 1$.

Case 1 to 3 are quite small with $|S| = 6, 7, 8$ and $k = 3$, so it is doable by hand or any brute force solution. This can be used to confirm your understanding of the problem.

Let us define $zig(i, j)$ to be the number of subsequences such that

- The length of the subsequences are j ,
- The i -th character of S is selected as the last character of the subsequence,
- Let T denote one of the subsequence, then we have $T_j < T_{j-1} > T_{j-2} < \dots$ (if you are wondering why we are defining the inequality from the back, you will see in a second)

Let's take "bbcade" for example, then we have $zig(4, 2) = 3$, because there are two subsequences "ba" and one "ca" that fulfills the property.

Similarly, we can define $zag(i, j)$, except that the inequalities are flipped for the third condition.

So what will be our answer? Our answer is $\sum (zig(i, k) + zag(i, k))$ for all i .

So now it remains to calculate $zig(i, k)$ and $zag(i, k)$ for all i .

We have this recurrence relation,

$$zig(i, j) = 1 \text{ if } j = 1$$

$$zig(i, j) = \sum zag(x, j - 1) \text{ otherwise } (j > 1)$$

for $x < i$ such that $S_x > S_i$

It is similar for $zag(i, j)$. The recurrence relation is left as an exercise to the reader.

Once we have this, we can define zig and zag as function in our program, and calculate the answer. But doing so directly will be very slow, as there are a lot of repeated function calls (ie. $zig(i, j)$ and $zag(i, j)$ can be called multiple times for a particular i, j). You might want to try running this once to see it.

To help with this, realise that once we have calculated the answer for a function call, we can simply store it, so that we can reuse the answer later. This technique is known as Dynamic Programming (DP), or more accurately memoization. For example, let `mzig` be a 2D array, such that `mzig[i][j]` is the result of $zig(i, j)$. Then, when you call $zig(i, j)$, first check if `mzig[i][j]` is already computed (you can initialise with `-1` to represent that $zig(i, j)$ has yet to be computed). If it is already computed, just return the value. If not, compute it (using the recurrence relation we found earlier) and then store it.

This will be good enough to quickly compute for case 4 - 7. For the full problem, we will still use Dynamic Programming, but we need to redefine our recurrence relation.

Define $zig(i, j, c)$ to be the number of subsequence such that

- The length of the subsequence is j ,
- Only the first i characters of S are considered (note that this is not the same having i -th character of S selected as the last character of the subsequence),
- The last character of the subsequence is c (c is one of the 26 lowercase English letters)
- Let T denote one of the subsequence, then we have $T_j < T_{j-1} > T_{j-2} < \dots$

We can define similarly for $zag(i, j, c)$ like before.

Once we have this, we can see that

$$zig(i, j, c) = \sum zag(i - 1, j - 1, c'), \text{ where } c' > c$$

and similarly for $zag(i, j, c)$. The full recurrence relation will be left as an exercise to the reader.

Verify that the answer to the question will then be $\sum(zig(|S|, k, c) + zag(|S|, k, c))$ for all possible last character c .

Once we have this, we then use Dynamic Programming as mentioned before, and this should be good enough for Case 8 - 10.

Below is one possible implementation which uses the same idea, but instead computes the answer bottom-up (rather than top down)

Implementation

```
mod = 1000
charSize = 26
s = "bvaycg"
kk = 3

zig = [[0] * charSize for i in range(kk + 1)]
zag = [[0] * charSize for i in range(kk + 1)]

zig[0][-1] = 1
zag[0][0] = 1

for cc in s:
    c = ord(cc) - ord('a')
    k = kk
    while k > 0:
        zig[k][c] = (zig[k][c] + sum(zag[k - 1][:c])) % mod
        zag[k][c] = (zag[k][c] + sum(zig[k - 1][(c + 1):])) % mod
        k -= 1

if k == 1:
    print(len(s) % mod)
else:
    print((sum(zig[kk]) + sum(zag[kk])) % mod)
```